TAREQO

16 — Migration W4 + H2 sur le Supabase de production (runbook)

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Étape 0 — Prérequis 3

Étape 1 — Sauvegarde (obligatoire) 3

Étape 2 — Pré-vol (lecture seule, SQL Editor) 4

Étape 3 — Appliquer `supabase/upgrade.sql` 4

- Option A — psql via le pooler, UNE transaction (recommandé — convention du projet) 4
- Option B — SQL Editor du Dashboard 4

Étape 4 — Vérification post-migration (lecture seule) 5

Étape 5 — Idempotence (optionnelle, confirme que la réapplication ne duplique pas) 5

Rollback (seulement si nécessaire) 5

Checklist finale 6

> Objectif : appliquer `supabase/upgrade.sql` au projet de production (dotklbpqmlnhcdifiukd) — amène la base EXISTANTE à l'état actuel : Partie 7 = W4 (argent en centimes entiers : NUMERIC → BIGINT) et Partie 8 = H2 (RPC get_cash_flow / get_sales_report + toutes les autres fonctions finales). C'étaient autrefois les migrations 010/011 ; aujourd'hui le script consolidé remplace toutes les migrations 002..011.
>
> Le script est 100% idempotent et a été validé 12/12 sur un vrai Postgres 16 (Docker, config CI exacte — scram/TCP), y compris la migration 25.50 → 2550 sans double application. Sans danger de le relancer autant de fois que nécessaire.
>
> Portugais (source faisant autorité) : `../pt/15-migracao-w4-h2.md`

Étape 0 — Prérequis

- Accès au Dashboard du projet + mot de passe de la base

(Settings → Database → Connection string).

- L'hôte direct du projet est IPv6 uniquement → utiliser le pooler

aws-0-eu-north-1.pooler.supabase.com.

- psql local ou Docker (postgres:16 embarque psql).
- Attention : NE JAMAIS exécuter scripts/sql_check.sh contre la base live — il

fait DROP SCHEMA public CASCADE. La vérification live passe uniquement par scripts/sql_check_live.sh (lecture seule).

Définir l'URL (remplacer SENHA) :

```
export
SUPABASE_DB_URL='postgresql://postgres.dotklbpqmlnhcdifiukd:SENHA@aws-0-eu-north-1.pooler.supabase.com:6543/
postgres?sslmode=require'
```

Étape 1 — Sauvegarde (obligatoire)

- Option rapide : Dashboard → Database → Backups → « Create a backup »

(ou activer le PITR).

- Option CLI (port 5432 *session* pooler, adapté à pg_dump) :

```
pg_dump
"postgresql://postgres.dotklbpqmlnhcdifiukd:SENHA@aws-0-eu-north-1.pooler.supabase.com:5432/postgres?sslmode=r
equire" \
```

```
--schema=public -f backup_pre_w4.sql
```

Étape 2 — Pré-vol (lecture seule, SQL Editor)

2.1 Types actuels des colonnes monétaires (si elles reviennent déjà en bigint, la migration a déjà été appliquée → passer à l'étape 4) :

```
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
 AND column_name IN ('purchase_price','sale_price','total_amount','unit_price','amount','price_monthly')
ORDER BY 1, 2;
```

2.2 Locataires en attente en double — la garde S3 (Partie 2 de upgrade.sql) interrompt le script s'il en existe. Vérifier d'abord :

```
SELECT owner_user_id, count(*)
FROM public.tenants
WHERE status = 'pending'
GROUP BY owner_user_id HAVING count(*) > 1;
```

→ Si la requête renvoie des lignes : résoudre les doublons (garder le plus ancien, rejeter les autres) avant d'appliquer.

2.3 Comptages de lignes (pour confirmer ensuite que rien n'a disparu) :

```
SELECT 'products' t, count(*) FROM public.products UNION ALL
SELECT 'sales', count(*) FROM public.sales UNION ALL
SELECT 'financial_transactions', count(*) FROM public.financial_transactions UNION ALL
SELECT 'debts', count(*) FROM public.debts;
```

Étape 3 — Appliquer supabase/upgrade.sql

Option A — psql via le pooler, UNE transaction (recommandé — convention du projet)

```
psql "$SUPABASE_DB_URL" -X -v ON_ERROR_STOP=1 -1 -f supabase/upgrade.sql
```

- -1 = transaction unique : si une quelconque partie échoue (ex. la garde S3), tout est annulé — aucun état partiel.
- Attendu : aucune erreur. En cas d'interruption, le message indique la raison (ex. doublons en attente) ; corriger et réappliquer.

Option B — SQL Editor du Dashboard

1. Ouvrir le SQL Editor du projet.
2. Coller le contenu complet de supabase/upgrade.sql.

3. Exécuter. L'éditeur exécute le script en un seul lot ; en cas d'erreur il

s'arrête et affiche le message. Comme le script est idempotent, corriger et relancer est sans danger.

Étape 4 — Vérification post-migration (lecture seule)

4.1 Exécuter le script de vérification live (lecture seule, 14 contrôles) :

```
SUPABASE_DB_URL="$SUPABASE_DB_URL" bash scripts/sql_check_live.sh
```

- Attendu : VERIFICAÇÃO CONCLUÍDA — todos os checks passaram (14/14),

couvrant : 9 colonnes monétaires en BIGINT, zéro valeur fractionnaire, RPC H2 présents (get_cash_flow/get_sales_report/get_dashboard_kpis) avec SECURITY DEFINER + retours BIGINT.

4.2 Contrôle manuel (SQL Editor) :

```
-- 1) Colonnes déjà en centimes (bigint) ?
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
 AND column_name IN ('purchase_price','sale_price','total_amount','unit_price','amount','price_monthly')
ORDER BY 1, 2;
-- Attendu : data_type = bigint sur les 9 colonnes.
-- 2) Plus aucune valeur fractionnaire ?
SELECT 'products' t, count(*) FROM public.products WHERE purchase_price % 1 <> 0 OR sale_price % 1 <> 0
UNION ALL SELECT 'sales', count(*) FROM public.sales WHERE total_amount % 1 <> 0
UNION ALL SELECT 'financial_transactions', count(*) FROM public.financial_transactions WHERE amount % 1 <> 0
UNION ALL SELECT 'debts', count(*) FROM public.debts WHERE amount % 1 <> 0;
-- Attendu : tout à 0.
-- 3) RPC H2 présents ?
SELECT p.proname, pg_get_function_identity_arguments(p.oid) args
FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
WHERE n.nspname = 'public'
 AND p.proname IN ('get_cash_flow','get_sales_report','get_dashboard_kpis')
ORDER BY 1;
```

4.3 Comptages de lignes — comparer avec l'étape 2.3 (rien ne doit manquer).

Étape 5 — Idempotence (optionnelle, confirme que la réapplication ne duplique pas)

Répéter l'étape 3, puis confirmer qu'il n'y a pas eu de double multiplication :

```
-- Un produit connu : ex. prix d'origine 25.50 doit être 2550 (une seule fois)
SELECT id, purchase_price, sale_price FROM public.products ORDER BY created_at LIMIT 3;
```

Rollback (seulement si nécessaire)

Restaurer les colonnes en NUMERIC(12,2) en divisant par 100 (les anciens RPC basés sur les reais cessent d'exister — l'app en mode Supabase retomberait sur l'agrégation côté client) :

```
ALTER TABLE public.products ALTER COLUMN purchase_price TYPE NUMERIC(12,2) USING (purchase_price / 100.0);
ALTER TABLE public.products ALTER COLUMN sale_price TYPE NUMERIC(12,2) USING (sale_price / 100.0);
-- ... répéter pour sales.total_amount, sale_items.unit_price,
-- financial_transactions.amount, debts.amount, purchases.total_amount,
-- purchase_items.unit_price, plans.price_monthly
```

Checklist finale

#	Élément	Vérification
1	Sauvegarde effectuée	le fichier backup_pre_w4.sql existe
2	Pré-vol propre	aucun doublon en attente ; types numeric
3	upgrade.sql appliqué	aucune erreur (transaction unique)
4	sql_check_live.sh	14/14
5	Réapplication sans effet	idempotence confirmée
6	App en mode Supabase	KPIs/rapports/cash flow avec les bonnes valeurs